

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Manual 05

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives.....	3
1 Game Playing in Artificial Intelligence.....	3
1.1 Characteristics of Game Playing Problems.....	3
1.2 Game State Representation.....	3
1.3 Utility (Evaluation) Function.....	4
2 Minimax Algorithm.....	4
2.1 Node Representation.....	5
3 Alpha-Beta Pruning for Minimax Algorithm.....	6
4 Python Libraries for Data Science.....	7
4.1 Pandas.....	7
• Dataset Handling.....	12
○ Significance of Data.....	12
○ Types of Data Sets.....	12
○ Mean, Median, Mode and Range of Data-Sets.....	13
○ Types of Attributes.....	14
○ Dataset Repositories.....	14
○ Properties of Attributes in a dataset.....	14
○ Iris Flower Dataset.....	15
○ Load the Dataset.....	16
• Visualization.....	17
3.1 Matplotlib.....	17

Objectives

After performing this lab, students shall be able to understand implementation of the following :

- ✓ Game Playing using minimax algorithm
- ✓ Revision of pandas and matplotlib

1 Game Playing in Artificial Intelligence

Game playing is one of the classical problems in Artificial Intelligence. In game-playing problems, an intelligent agent must make decisions by considering possible future moves and the responses of an opponent.

These problems are usually modeled as:

- Two-player
- Turn-based
- Zero-sum games

Examples include:

- Tic-Tac-Toe
- Chess
- Checkers
- Connect Four

1.1 Characteristics of Game Playing Problems

- There are two players:
 - MAX (the AI or computer)
 - MIN (the opponent)
- Players take turns
- One player's gain is the other player's loss
- Both players are assumed to play optimally

1.2 Game State Representation

In game playing:

- A state represents a configuration of the game board
- The initial state is the starting position
- Terminal states are game-ending positions (win, lose, or draw)
- Each state has possible actions leading to new states

The game is represented as a game tree.

2.2.1 Game Tree

A game tree is a tree structure where:

- Each node represents a game state
- Each edge represents a possible move
- Levels of the tree alternate between MAX and MIN players
- MAX nodes: try to maximize the score
- MIN nodes: try to minimize the score

1.3 Utility (Evaluation) Function

At terminal states, a utility function assigns a numeric value indicating the outcome of the game.

Typical utility values:

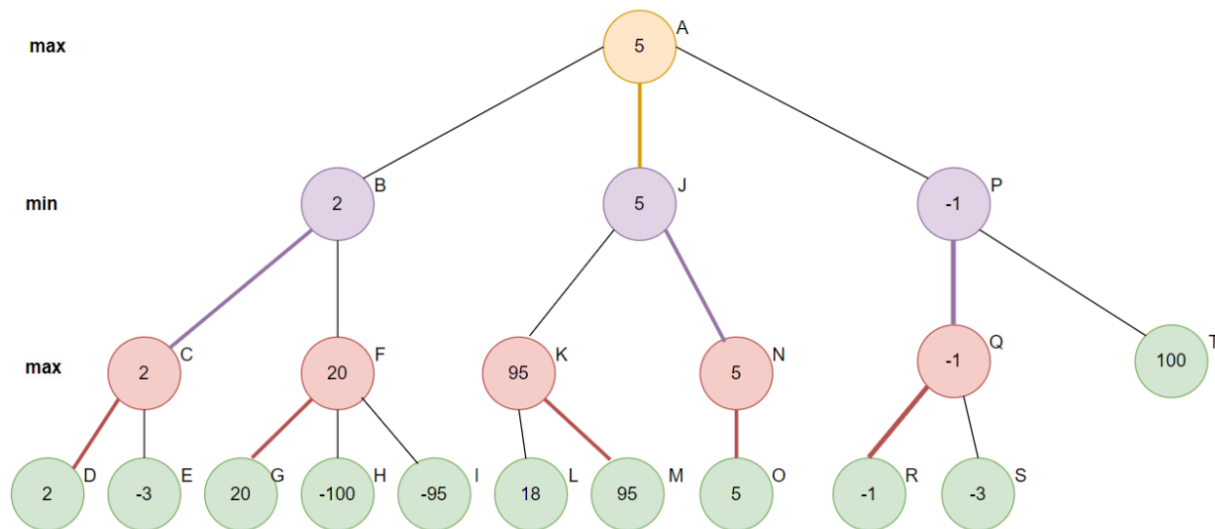
- Win for MAX: +1 (or positive value)
- Loss for MAX: -1 (or negative value)
- Draw: 0

2 Minimax Algorithm

The Minimax algorithm is a recursive decision-making algorithm used for optimal play in two-player games.

It assumes:

- Both players play optimally
- MAX tries to maximize the utility
- MIN tries to minimize the utility



Algorithm:

- Start from the root node
- If the node is a terminal state:
 - Return its utility value
- If it is MAX's turn:
 - Evaluate all child nodes
 - Return the maximum value
- If it is MIN's turn:
 - Evaluate all child nodes
 - Return the minimum value

```
MINIMAX(node, depth, isMaxPlayer):
```

```
  if node is terminal or depth == 0:
    return utility(node)
```

```
  if isMaxPlayer:
```

```
    bestValue =  $-\infty$ 
```

```
    for each child of node:
```

```
      value = MINIMAX(child, depth-1, False)
```

```
      bestValue = max(bestValue, value)
```

```
    return bestValue
```

```
  else:
```

```
    bestValue =  $+\infty$ 
```

```
    for each child of node:
```

```
      value = MINIMAX(child, depth-1, True)
```

```
      bestValue = min(bestValue, value)
```

```
    return bestValue
```

2.1 Node Representation

A node in a game tree may contain:

- Game state
- List of possible moves
- Player type (MAX or MIN)
- Utility value (for terminal nodes)

For example:

```
class Node:
    def __init__(self, value=None, children=None):
        self.value = value
        self.children = children
```

3 Alpha-Beta Pruning for Minimax Algorithm

The basic Minimax algorithm explores all possible nodes in the game tree. For games with large branching factors, this becomes computationally expensive.

Alpha-Beta Pruning improves Minimax by:

- Avoiding exploration of branches that cannot affect the final decision
- Reducing the number of nodes evaluated
- Producing the same result as Minimax but faster

Alpha-Beta Pruning keeps track of two values during the search:

- α (alpha): The best value MAX can guarantee so far
- β (beta): The best value MIN can guarantee so far
- If at any point: $\alpha \geq \beta$ the remaining branches do not need to be explored because they cannot influence the final outcome.

Alpha (α) is used at MAX nodes. It is initially set to $-\infty$ and only updated when MAX finds a better value. Similarly Beta (β) is used at MIN nodes. It is initially set to $+\infty$ and updated only when MIN finds a better value

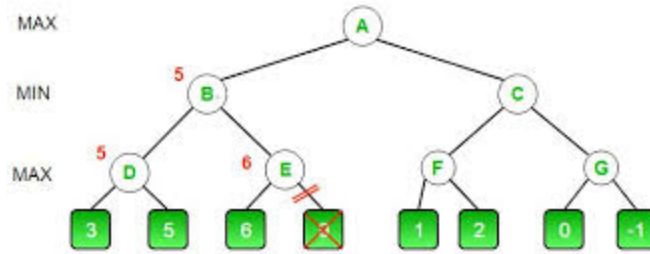
Pruning occurs:

- At a MAX node: If current value $\geq \beta$, stop exploring further children (beta cut-off)
- At a MIN node: If current value $\leq \alpha$, stop exploring further children (alpha cut-off)

These cut-offs are called pruning.

Algorithm Steps

- Start from the root node
- Pass α and β values along with each recursive call
- Update α at MAX nodes and β at MIN nodes
- Prune branches when $\alpha \geq \beta$
- Return the optimal value



4 Python Libraries for Data Science

4.1 Pandas

The Pandas library is one of the most preferred tools for data scientists to do data manipulation and analysis. It can work with data from a wide variety of sources. Pandas is suited for many different kinds of data: tabular data, time-series data, arbitrary matrix data with row and column labels, and any other form of observational/statistical data sets.

Features:

- Enables you to create your own function and run it across a series of data
- Contains high-level data structures and manipulation tools

■ Pandas Installation

• Installation via Terminal

To install pandas in your system you can use this command `pip install pandas` or `conda install pandas`.

• Installation in Anaconda

Pandas or any other packages can be installed in Anaconda environment by following the tutorial [here](#).

Import Convention

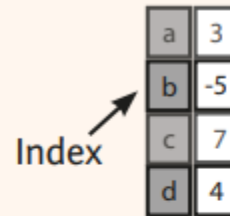
```
>>> import pandas as pd
```

■ Series

Pandas series works the same way both in list and numpy array as well as dictionary. To make series in pandas we need to use `pd.Series(data, index)` format where `data` are input data and `index` are selected index for data. A one-dimensional labeled array capable of holding any data type:

Series

A one-dimensional labeled array capable of holding any data type



a	3
b	-5
c	7
d	4

```
>>> s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
```

```
>>> s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
```

A	3
B	5
C	7
D	4

Pandas series along with numPy array:

```
import numpy as np          #importing numpy
import pandas as pd         #importing pandas
arr=np.array([1,3,5,7,9])   #create arr array
s2=pd.Series(arr)           #create pandas series s2
print(s2)                  #print s2
print(type(s2))            #print type of s2
```

Output:

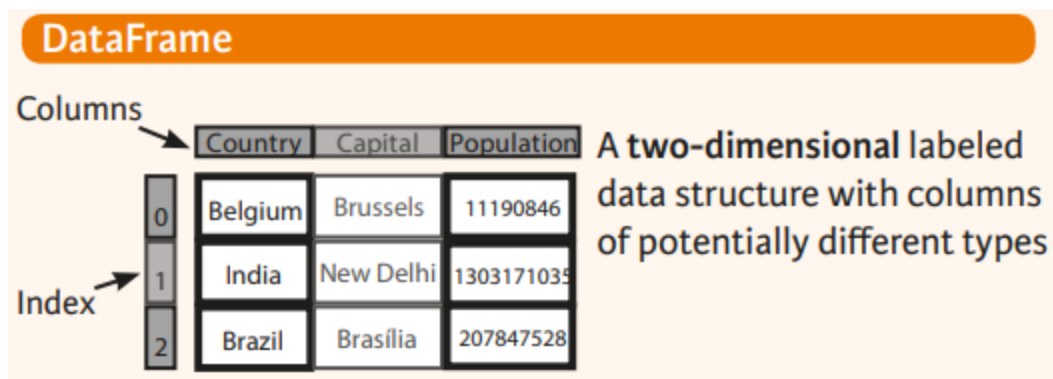
```
0    1
1    3
2    5
3    7
4    9
dtype: int64
<class 'pandas.core.series.Series'>
```

■ DataFrame

Pandas DataFrame is a way to store data in rectangular grids that can easily be overviewed. It's like a tabular data structure with labeled axes (rows and columns). The default format of a DataFrame would be `pd.DataFrame(data, index, column)`. You need to mention the data, index and columns value to generate a DataFrame. Data should be at least *two-dimensional*, *index* will be the row name and *columns* values for the columns. In general, you could say that the Pandas DataFrame consists of three main components: the data, the index, and the columns.

The DataFrame can contain data that is:

- Pandas DataFrame
- Pandas Series: An example of a Series object is one column from a DataFrame.
- a NumPy ndarray, which can be a record or structured
- a two-dimensional ndarray
- lists, dictionaries or Series.



```
>>> data = {'Country': ['Belgium', 'India', 'Brazil'],
'Capital': ['Brussels', 'New Delhi', 'Brasilia'],
'Population': [11190846, 1303171035, 207847528]}
>>> df = pd.DataFrame(data, columns=['Country', 'Capital', 'Population'])
```

To get subset of a Data Frame:

```
>>> df[1:]
Country    Capital    Population
1  India      New Delhi  1303171035
2  Brazil    Brasilia   207847528
```

```
import pandas as pd
s2 = pd.Series([10, 20, 30])
print(s2)
print(type(s2))
s3=pd.DataFrame([[1,2],[3,4]],columns=['A','B'], index = ['C', 'D'])
print(s3)
print(type(s3))
```

```
0    10
1    20
2    30
dtype: int64
<class 'pandas.core.series.Series'>
   A  B
C  1  2
D  3  4
<class 'pandas.core.frame.DataFrame'>
```

Figure 7 Example of Pandas Series and DataFrame

■ Indexing

- `.loc[]` works on labels of your index. This means that if you give in `loc[2]`, you look for the values of your DataFrame that have an index labeled 2.
- `.iloc[]` works on the positions in your index. This means that if you give in `iloc[2]`, you look for the values of your DataFrame that are at index '2'.

■ Input Output Operations

Read and Write to CSV

```
pd.read_csv('file.csv', header=None, nrows=5)
df.to_csv('myDataFrame.csv')
```

■ Other Operations

- Set index a of Series s to 6

```
>>> s['a'] = 6
```

- Sort by the values along an axis

```
>>> df.sort_values(by='Country')
```

- Assign ranks to entries

```
>>> df.rank()
```

■ Retrieving Series/DataFrame Information

- Basic Information (rows, columns)

```
>>> df.shape
```

- Describe index

```
>>> df.index
```

- Describe DataFrame columns

```
>>> df.columns
```

- Info on DataFrame

```
>>> df.info()
```

- Number of non-NA values

```
>>> df.count()
```

- Sum of values

```
>>> df.sum()
```

- Cumulative sum of values

```
>>> df.cumsum()
```

- Minimum/maximum values

```
>>> df.min()/df.max()
```

- Minimum/Maximum index value

```
>>> df.idxmin()/df.idxmax()
```

■ Application of Functions

```
>>> f = lambda x: x*2
```

Apply function

```
>>> df.apply(f)
```

Find more details about Pandas [here](#).

- **Dataset Handling**

- **Significance of Data**

A data set is a set or collection of data. This set is normally presented in a tabular pattern. Every column describes a particular variable. And each row corresponds to a given member of the data set, as per the given question. The data are essentially organized to a certain model that helps to process the needed information. This set of data is any permanently saved collection of information that usually contains either case-level, gathered data, or statistical guidance level data.

In Machine Learning projects, it is impossible for an “AI” to learn without data. During an AI development, we always rely on data. From training, tuning, model selection to testing, we use three different data sets: the training set, the validation set and the testing set.

- **Training data set:** The training data set is the one used to train an algorithm to understand how to apply concepts such as neural networks, to learn and produce results. It is the actual **data set** used to train the model for performing various actions. It includes both input data and the expected output.
- **Test data set:** The test data set is used to evaluate how well your algorithm was trained with the training data set. In AI projects, we can't use the training data set in the testing stage because the algorithm will already know in advance the expected output which is not our goal.

- **Types of Data Sets**

In Statistics, we have different types of data sets available for different types of information. They are:

- Numerical data sets
- Bivariate data sets
- Multivariate data sets
- Categorical data sets
- Correlation data sets

Let us discuss all these data sets with examples.

- **Numerical Data Sets**

A set of all numerical data. It deals only with numbers. Some of the examples are;

- Weight and height of a person

- The count of RBC in a medical report
- Number of pages present in a book

■ Bivariate Data Sets

A data set that has two variables is called a Bi-variate data set. It deals with the relationship between the two variables.

Example: To find the percentage score and age of the students in a class. Score and age can be considered as two variables.

■ Multivariate Data Sets

A data set with multiple variables.

Example: If we have to measure the length, width, height, volume of a rectangular box, we have to use multiple variables to distinguish between those entities.

■ Categorical Data Sets

Categorical data sets represent features or characteristics of a person or an object.

Example:

- A person's gender (male or female)
- Marital status (married/unmarried)

■ Correlation Data Sets

The set of values that demonstrate some relationship with each other indicates correlation data sets. Here the values are found to be dependent on each other.

Example: A tall person is considered to be heavier than a short person. So here the weight and height variables are dependent on each other.

○ Mean, Median, Mode and Range of Data-Sets

The **mean, median and mode** along with range are the major topics in Statistics. Let us get through with respect to data-sets here.

Mean of a data-set is the average of all the observations present in the table. It is the ratio of the sum of observations to the total number of elements present in the data-set. The formula of mean is given by;

$$\text{Mean} = \text{Sum of Observations} / \text{Total Number of Elements in Data Set}$$

Median of a data-set is the middle value of the collection of data when arranged in ascending order and descending order.

Mode of a data-set is the variable or number or value which is repeated maximum number of times in the set.

Range of a data set is the difference between the maximum value and minimum value.

Range = Maximum Value – Minimum Value

Example:

Find the mean, mode, median and range of the given data set.

{2, 4, 6, 8, 2, 10, 12}

Solution:

Given, {2, 4, 6, 8, 2, 10, 12} is a set of data.

Mean = $2+4+6+8+2+10+12/7 = 44/7$

To find median we have to first arrange the given data in ascending or descending order

So, {2,2,4,6,8,10,12}. Thus,

Median = 6

Mode = 2

Range = $12-2 = 10$

○ Types of Attributes

- **Nominal**

Examples: ID numbers, eye color, zip codes

- **Ordinal**

Examples: rankings (e.g., taste of potato chips on a scale from 1-10), grades, height in {tall, medium, short}

- **Interval**

Examples: calendar dates, temperatures in Celsius or Fahrenheit.

- **Ratio**

Examples: temperature in Kelvin, length, time, counts

○ Dataset Repositories

Datasets for analysis can be downloaded from common repositories like:

✓ Kaggle

✓ UCI

Check out [this](#) & [this](#) links for a list of various data repositories.

○ Properties of Attributes in a dataset

■ Missing Values

Some entries can be missing because of the following reasons:

- 5 **Data Extraction:** It is possible that there are problems with extraction process. In such cases, we should double-check for correct data with data guardians. Errors at data extraction stage are typically easy to find and can be corrected easily as well.
- 6 **Data collection:** These errors occur at time of data collection and are harder to correct.

■ Outliers

Outlier is an observation that appears far away and diverges from an overall pattern in a sample. Let's take an example, we do customer profiling and find out that the average annual income of customers is \$1 lakh. But, there are two customers having annual income of \$4 and \$4.2 million. These two customers annual income is much higher than rest of the population. These two observations will be seen as Outliers.

Outliers can be due to the following reasons:

- **Data Entry Errors:** Human errors such as errors caused during data collection, recording, or entry can cause outliers in data. For example: Annual income of a customer is \$100,000. Accidentally, the data entry operator puts an additional zero in the figure. Now the income becomes \$1,000,000 which is 10 times higher. Evidently, this will be the outlier value when compared with rest of the population.
- **Measurement Error:** It is the most common source of outliers. This is caused when the measurement instrument used turns out to be faulty. For example: There are 10 weighing machines. 9 of them are correct, 1 is faulty. Weight measured by people on the faulty machine will be higher / lower than the rest of people in the group. The weights measured on faulty machine can lead to outliers.
- **Experimental Error:** Another cause of outliers is experimental error. For example: In a 100m sprint of 7 runners, one runner missed out on concentrating on the 'Go' call which caused him to start late. Hence, this caused the runner's run time to be more than other runners. His total run time can be an outlier.
- **Data Processing Error:** Whenever we perform data mining, we extract data from multiple sources. It is possible that some manipulation or extraction errors may lead to outliers in the dataset.
- **Sampling error:** For instance, we have to measure the height of athletes. By mistake, we include a few basketball players in the sample. This inclusion is likely to cause outliers in the dataset.

○ Iris Flower Dataset

The Iris flower data set or Fisher's Iris data set is a multivariate data set introduced by the British statistician. The dataset contains 150 observations of iris flowers. There are four columns of measurements of the flowers in centimeters. The fifth column is the species of the flower observed. All observed flowers belong to one of three species. The data set consists of 50 samples from each of three species of Iris (Iris setosa, Iris virginica and Iris versicolor).

To gain more insights about this dataset check [this link](#).

You can download this dataset from [here](#).

○ Load the Dataset

In order to load the dataset, run Anaconda Navigator.

- Launch Jupyter Notebook
- Click on “Upload” button as shown in fig. 10
- Select the dataset file from your directory
- Create a new Python3 notebook by selecting “New” on the Jupyter host page
- Follow the instructions given in the code snippet provided next



Figure 8 Load Dataset in Anaconda

Open the newly created notebook and implement the code given below:

```
import pandas as pd

iris_dataset= pd.read_csv("iris.csv")
iris_dataset
```

Jupyter Load Iris Data Last Checkpoint: 19 hours ago (autosaved)

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3

```
In [2]: import pandas as pd
```

```
In [3]: iris_dataset = pd.read_csv("iris.csv")
```

```
In [4]: iris_dataset
```

Out[4]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
5	6	5.4	3.9	1.7	0.4	Iris-setosa
6	7	4.6	3.4	1.4	0.3	Iris-setosa
7	8	5.0	3.4	1.5	0.2	Iris-setosa
8	9	4.4	2.9	1.4	0.2	Iris-setosa
9	10	4.9	3.1	1.5	0.1	Iris-setosa

For more information about file handling using Pandas, visit this [site](#).

• Visualization

3.1 Matplotlib

Matplotlib is a low-level graph plotting library in python that serves as a visualization utility. Matplotlib was created by John D. Hunter. Matplotlib is open source and we can use it freely. Matplotlib is mostly written in python, a few segments are written in C, Objective-C and JavaScript for Platform compatibility.

3.1 Installation of Matplotlib

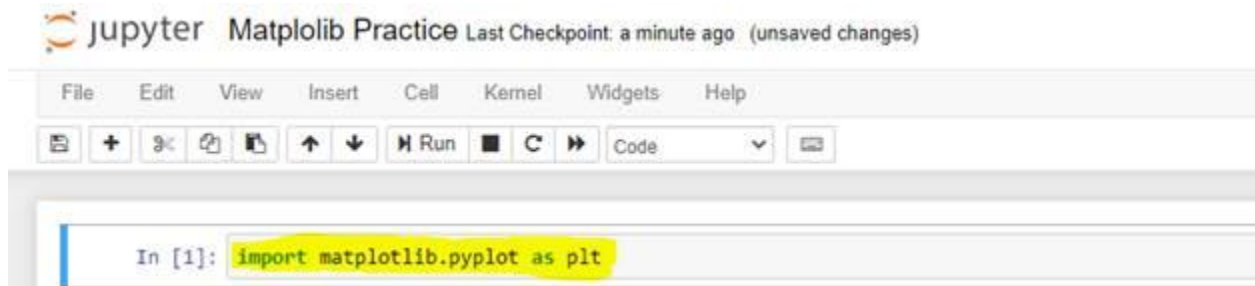
If you have Python and PIP already installed on a system, then installation of Matplotlib is very easy.

```
pip install matplotlib
```

But mostly distribution like Anaconda, Spyder have pre-installed matplotlib.

3.2 Pyplot

Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias:

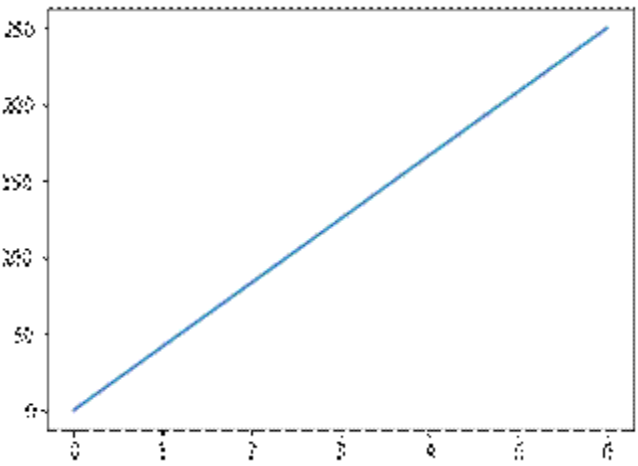


```
import matplotlib.pyplot as plt
```

Now the Pyplot package can be referred to as plt.

Example

Draw a line in a diagram from position (0,0) to position (6,250):

Code	Output
<pre>import matplotlib.pyplot as plt import numpy as np xpoints=np.array([0, 6]) ypoints=np.array([0, 250]) plt.plot(xpoints, ypoints) plt.show()</pre>	 The output is a line plot with a light blue line. The x-axis is labeled from 0 to 6 with major ticks every 1 unit. The y-axis is labeled from 0 to 250 with major ticks every 50 units. The line starts at the origin (0,0) and extends linearly to the point (6,250).

3.3 Plotting x and y points

The `plot()` function is used to draw points (markers) in a diagram.

By default, the `plot()` function draws a line from point to point.

The function takes parameters for specifying points in the diagram.

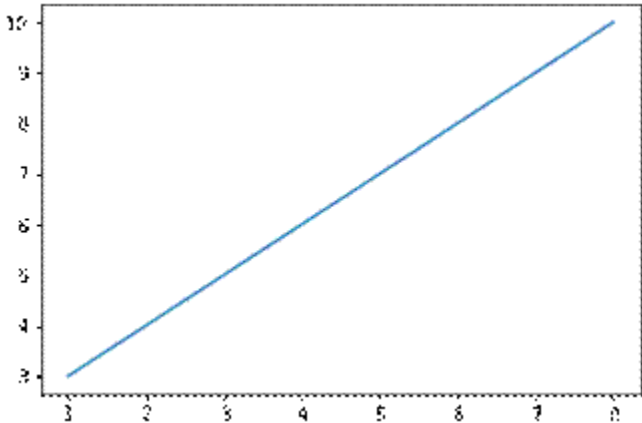
Parameter 1 is an array containing the points on the **x-axis**.

Parameter 2 is an array containing the points on the **y-axis**.

If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the `plot` function.

Example

Draw a line in a diagram from position (1, 3) to position (8, 10):

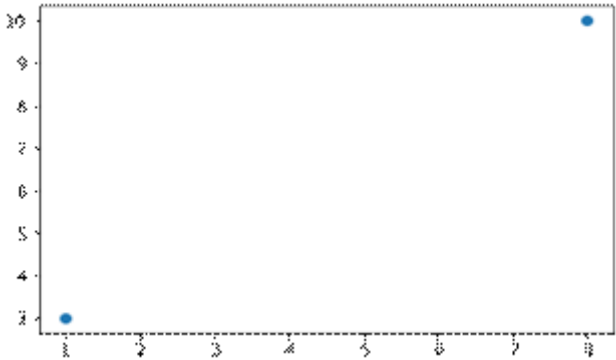
Code	Output
<pre>import matplotlib.pyplot as plt import numpy as np xpoints = np.array([1, 8]) ypoints = np.array([3, 10]) plt.plot(xpoints, ypoints) plt.show()</pre>	

There are many types of single lines/multiple lines that can be drawn, explore other types at: https://www.w3schools.com/python/matplotlib_line.asp

3.4 Plotting Without Line

To plot only the markers, you can use *shortcut string notation* parameter 'o', which means 'rings'.

Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

Code	Output
<pre>import matplotlib.pyplot as plt import numpy as np xpoints = np.array([1, 8]) ypoints = np.array([3, 10]) plt.plot(xpoints, ypoints, 'o') plt.show()</pre>	

There can be different type of markers, you can explore at:

https://www.w3schools.com/python/matplotlib_markers.asp

3.5 Multiple Points

You can plot as many points as you like, just make sure you have the same number of points in both axis.

Example

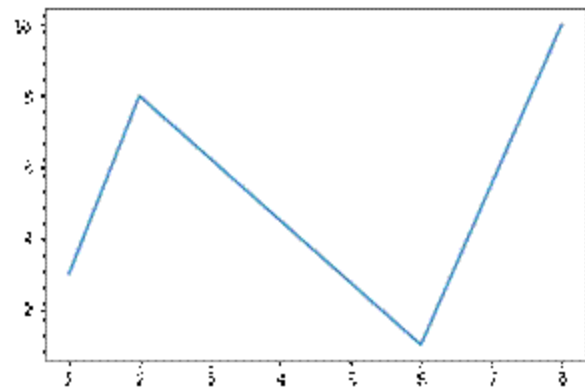
Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

Code	Output
------	--------

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])

plt.plot(xpoints, ypoints)
plt.show()
```



3.6 Default X-Points

If we do not specify the points in the x-axis, they will get the default values 0, 1, 2, 3, (etc. depending on the length of the y-points).

So, if we take the same example as above, and leave out the x-points, the diagram will look like this:

Example

Plotting without x-points:

Code	Output														
<pre>import matplotlib.pyplot as plt import numpy as np ypoints = np.array([3, 8, 1, 10, 5, 7]) plt.plot(ypoints) plt.show()</pre>	<table border="1"> <thead> <tr> <th>xpoints</th> <th>ypoints</th> </tr> </thead> <tbody> <tr> <td>0</td> <td>3</td> </tr> <tr> <td>1</td> <td>8</td> </tr> <tr> <td>2</td> <td>1</td> </tr> <tr> <td>3</td> <td>10</td> </tr> <tr> <td>4</td> <td>5</td> </tr> <tr> <td>5</td> <td>7</td> </tr> </tbody> </table>	xpoints	ypoints	0	3	1	8	2	1	3	10	4	5	5	7
xpoints	ypoints														
0	3														
1	8														
2	1														
3	10														
4	5														
5	7														

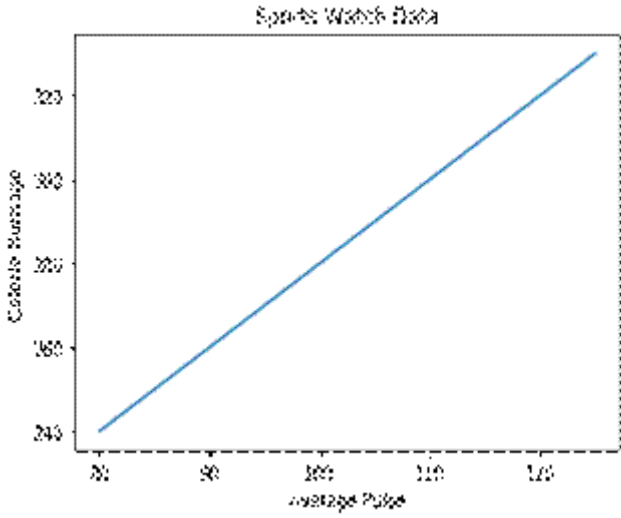
The x-points in the example above are [0, 1, 2, 3, 4, 5] by default.

3.7 Create Labels and title for a Plot

With Pyplot, you can use the `xlabel()` and `ylabel()` functions to set a label for the x- and y-axis.

Example

Add labels to the x- and y-axis:

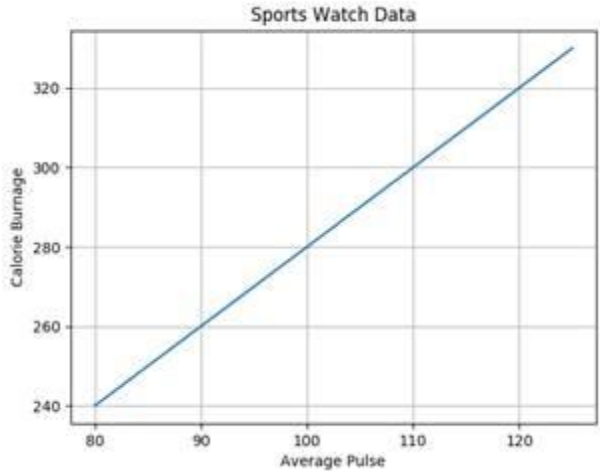
Code	Output
<pre>import numpy as np import matplotlib.pyplot as plt x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125]) y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330]) plt.plot(x, y) plt.title("Sports Watch Data") plt.xlabel("Average Pulse") plt.ylabel("Calorie Burnage") plt.show()</pre>	

3.8 Add Grid Lines to a Plot

With Pyplot, you can use the `grid()` function to add grid lines to the plot.

Example

Add grid lines to the plot:

Code	Output																						
<pre> import numpy as np import matplotlib.pyplot as plt x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125]) y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330]) plt.title("Sports Watch Data") plt.xlabel("Average Pulse") plt.ylabel("Calorie Burnage") plt.plot(x, y) plt.grid() plt.show()</pre>	 <table border="1"> <caption>Data points from the Sports Watch Data plot</caption> <thead> <tr> <th>Average Pulse</th> <th>Calorie Burnage</th> </tr> </thead> <tbody> <tr><td>80</td><td>240</td></tr> <tr><td>85</td><td>250</td></tr> <tr><td>90</td><td>260</td></tr> <tr><td>95</td><td>270</td></tr> <tr><td>100</td><td>280</td></tr> <tr><td>105</td><td>290</td></tr> <tr><td>110</td><td>300</td></tr> <tr><td>115</td><td>310</td></tr> <tr><td>120</td><td>320</td></tr> <tr><td>125</td><td>330</td></tr> </tbody> </table>	Average Pulse	Calorie Burnage	80	240	85	250	90	260	95	270	100	280	105	290	110	300	115	310	120	320	125	330
Average Pulse	Calorie Burnage																						
80	240																						
85	250																						
90	260																						
95	270																						
100	280																						
105	290																						
110	300																						
115	310																						
120	320																						
125	330																						

Different type of grid can be generated, for more details see:

https://www.w3schools.com/python/matplotlib_grid.asp

3.9 Display Multiple Plots

With the `subplots()` function you can draw multiple plots in one figure:

Example

Draw 2 plots:

Code	Output
------	--------

```
import matplotlib.pyplot as plt
import numpy as np
```

```
#plot 1:
```

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
```

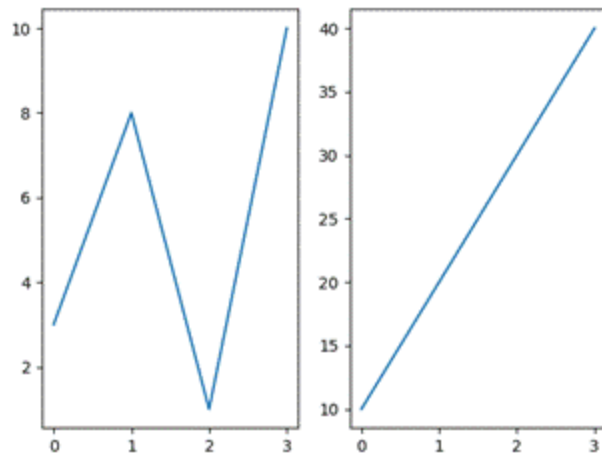
```
plt.subplot(1, 2, 1)
plt.plot(x,y)
```

```
#plot 2:
```

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(1, 2, 2)
plt.plot(x,y)
```

```
plt.show()
```



There different ways to plot multiple plots:

https://www.w3schools.com/python/matplotlib_subplots.asp

3.10 Creating Scatter Plots

With Pyplot, you can use the `scatter()` function to draw a scatter plot.

The `scatter()` function plots one dot for each observation. It needs two arrays of the same length, one for the values of the x-axis, and one for values on the y-axis:

Example:

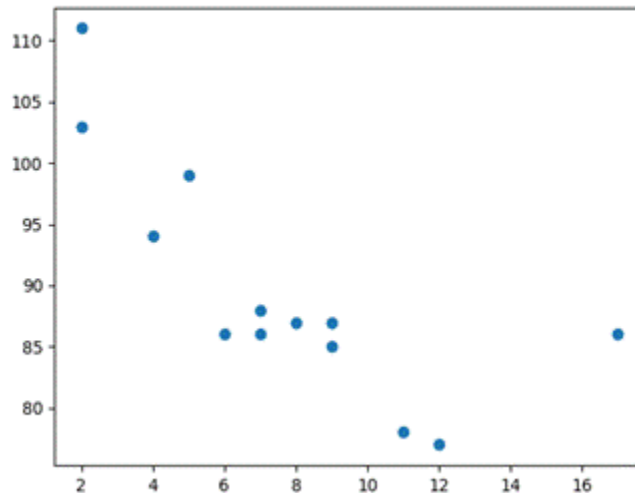
Code	Output
------	--------

```
import matplotlib.pyplot as plt
import numpy as np

x=np.array([5,7,8,7,2,17,2,9,
4,11,12,9,6])

y=np.array([99,86,87,88,111,
86,103,87,94,78,77,85,86])

plt.scatter(x,y)
plt.show()
```



Explanation of above plot:

The observation in the example above is the result of 13 cars passing by. The X-axis shows how old the car is. The Y-axis shows the speed of the car when it passes. Are there any relationships between the observations? It seems that the newer the car, the faster it drives, but that could be a coincidence, after all we only registered 13 cars.

There are different type of scatter graphs that can be created (kindly see the link given, as all examples will make the manual lengthy):

https://www.w3schools.com/python/matplotlib_scatter.asp

3.11 Creating Bars

With Pyplot, you can use the `bar()` function to draw bar graphs:

Example

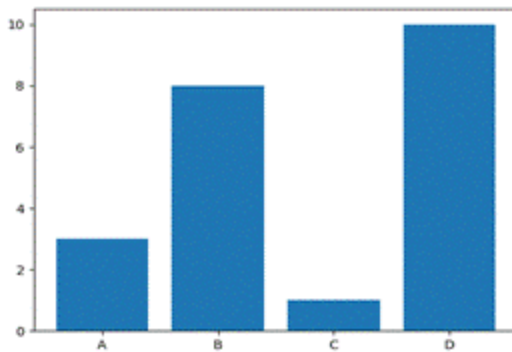
Draw 4 bars:

Code	Output
------	--------

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
plt.show()
```

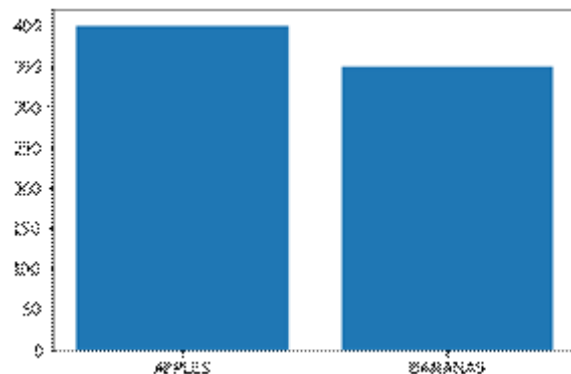


The `bar()` function takes arguments that describes the layout of the bars.

The categories and their values represented by the *first* and *second* argument as arrays.

```
import matplotlib.pyplot as plt

x = ["APPLES", "BANANAS"]
y = [400, 350]
plt.bar(x, y)
```

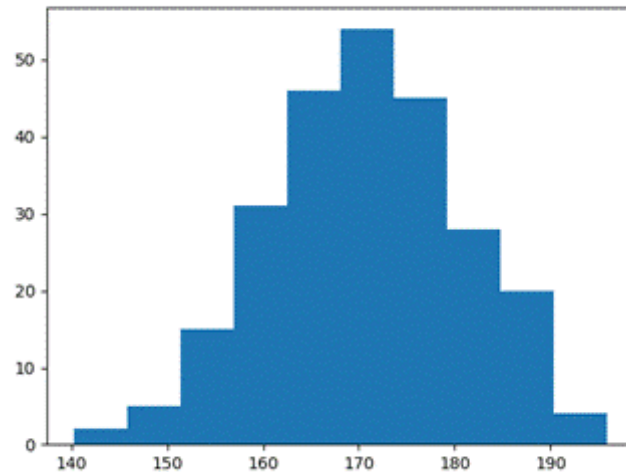


3.12 Histogram

A histogram is a graph showing *frequency* distributions. It is a graph showing the number of observations within each given interval. Example: Say you ask for the height of 250 people; you might end up with a histogram like this:

You can read from the histogram that there are approximately:

2 people from 140 to 145cm
 5 people from 145 to 150cm
 15 people from 151 to 156cm
 31 people from 157 to 162cm
 46 people from 163 to 168cm
 53 people from 168 to 173cm
 45 people from 173 to 178cm
 28 people from 179 to 184cm
 21 people from 185 to 190cm
 4 people from 190 to 195cm



3.12.1 Create Histogram

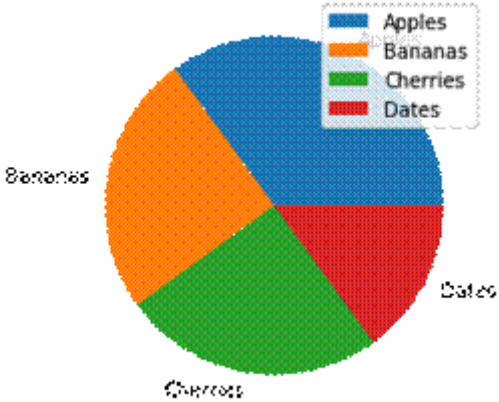
In Matplotlib, we use the `hist()` function to create histograms.

The `hist()` function will use an array of numbers to create a histogram, the array is sent into the function as an argument. For simplicity we use NumPy to randomly generate an array with 250 values, where the values will concentrate around 170, and the standard deviation is 10.

Code	Output
<pre>import matplotlib.pyplot as plt import numpy as np x = np.random.normal(170, 10, 250) plt.hist(x) plt.show()</pre>	

3.13 Creating Pie Charts

With Pyplot, you can use the `pie()` function to draw pie charts:

Code	Output
<pre>import matplotlib.pyplot as plt import numpy as np y = np.array([35, 25, 25, 15]) mylabels = ["Apples", "Bananas", "Cherries", "Dates"] plt.pie(y, labels = mylabels) plt.legend() plt.show()</pre>	

As you can see the pie chart draws one piece (called a wedge) for each value in the array (in this case [35, 25, 25, 15]).

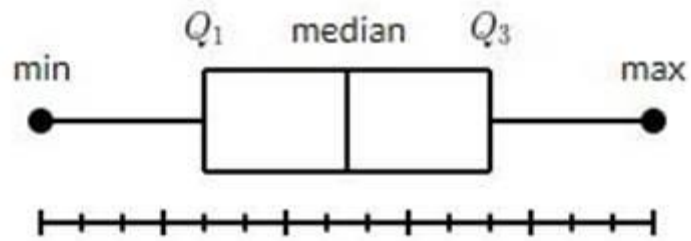
By default, the plotting of the first wedge starts from the x-axis and move *counterclockwise*:

Note: The size of each wedge is determined by comparing the value with all the other values, by using this formula:

The value divided by the sum of all values: $x/\text{sum}(x)$

3.14 Box Plot

A box plot which is also known as a whisker plot displays a summary of a set of data containing the minimum, first quartile, median, third quartile, and maximum. In a box plot, we draw a box from the first quartile to the third quartile. A vertical line goes through the box at the median. The whiskers go from each quartile to the minimum or maximum.



The image is taken from: https://www.tutorialspoint.com/matplotlib/matplotlib_box_plot.htm

Example 1: Draw a box-and-whisker plot for the data set {3, 7, 8, 5, 12, 14, 21, 13, 18}.

Minimum: 3, Q_1 : 6, Median: 12, Q_3 : 16, and Maximum: 21.

Code	Output
<pre>import matplotlib.pyplot as plt data = [3, 7, 8, 5, 12, 14, 21, 13, 18] plt.boxplot(data) plt.show()</pre>	